

Project Planning Phase

Project Planning Template (Product Backlog, Sprint Planning, Stories, Story points)

Date	28 September 2025
Team ID	SWTID-2026-2056
Project Name	Implement Client Script & UI Policy (Incident)
Maximum Marks	5 Marks

Product Backlog, Sprint Schedule, and Estimation (4 Marks)

Use the below template to create product backlog and sprint schedule

Sprint	Functional Requirement (Epic)	User Story Number	User Story / Task	Story Points	Priority	Team Members
Sprint-1	UI Policy – Field Behaviour	USN-1	As an agent, when I set Incident State to Resolved, Close notes becomes mandatory, Opened by is hidden, and Priority/Urgency become read-only.	3	High	
Sprint-1		USN-2	As an agent, when State changes away from Resolved, field attributes reverse automatically (Reverse if false).	2	High	
Sprint-1	Client Script – onChange	USN-3	As an agent, when I change Priority, the field background colour updates (Critical=red ... Planning=green).	2	Medium	
Sprint-1	Client Script – onSubmit	USN-4	As an agent, I cannot submit/resolve an Incident without filling Close notes when State is Resolved.	3	High	
Sprint-1	Client Script – Auto-populate	USN-5	As an agent, when I select a Caller, Location and Contact number auto-populate from the user record.	3	High	
Sprint-1	Configuration	USN-6	As an administrator, I can create and activate a UI Policy on the Incident table with conditions and UI Policy Actions.	2	High	
Sprint-1		USN-7	As an administrator, I can create onLoad, onChange and onSubmit Client Scripts for the Incident table and set UI Type.	3	High	
Sprint-2	UI Policy – Conditional Visibility	USN-8	As a manager, when an Incident is marked Major Incident, additional fields become visible and mandatory.	3	High	
Sprint-2	Restricted Form View	USN-9	As a caller viewing my own Incident, technical fields (Assignment group, Work notes) are hidden or read-only via UI Policy.	2	Medium	
Sprint-2	Client Script – Server Interaction	USN-10	As an agent, Client Scripts use GlideAjax / getReference to fetch additional data and update the form asynchronously.	3	Medium	
Sprint-2	Best-Practice Decision	USN-11	As an administrator, I choose UI Policy for simple declarative control and Client Script for complex logic / onSubmit validation.	2	Medium	
Sprint-2	Testing & Documentation	USN-12	As a developer, I test all scripts/policies on Desktop and Service Portal and document naming conventions and order of execution.	3	High	
Sprint-3	Advanced UI Policy	USN-13	As an administrator, I implement view-specific UI Policies and related-list visibility controls based on State/Category.	3	Medium	
Sprint-3	Client Script – onLoad	USN-14	As an agent, on form load the script sets initial attributes, shows contextual messages and pre-populates fields for existing records.	2	Medium	
Sprint-3	Security Alignment	USN-15	As an administrator, critical rules are also enforced via ACLs and Data Policies so they cannot be bypassed client-side.	3	High	
Sprint-3	Performance Tuning	USN-16	As a developer, I minimise onChange scripts on high-frequency fields and prefer async GlideAjax for server calls.	2	Medium	

Project Tracker, Velocity & Burndown Chart: (4 Marks)

Sprint	Total Story Points	Duration	Sprint Start Date	Sprint End Date (Planned)	Story Points Completed (as on Planned End Date)	Sprint Release Date (Actual)
Sprint-1	18	6 Days	06 Oct 2025	11 Oct 2025	18	11 Oct 2025
Sprint-2	13	6 Days	13 Oct 2025	18 Oct 2025	13	18 Oct 2025
Sprint-3	10	6 Days	20 Oct 2025	25 Oct 2025		
Sprint-4	Buffer / Hardening	4 Days	27 Oct 2025	30 Oct 2025		

Velocity:

The team's planned velocity is based on a 6-day sprint duration. Total story points committed across the main development sprints:

Sprint-1: 18 points | Sprint-2: 13 points | Sprint-3: 10 points → Total committed ≈ 41 points

Average velocity (AV) per day (using Sprint-1 as baseline example of 18 points in 6 days):

$$\text{AV} = \text{Velocity} / \text{Sprint Duration} = 18 / 6 = 3 \text{ story points per day}$$

(If a 10-day sprint with velocity 20 is used for illustration: $AV = 20 / 10 = 2$ story points per day.)

Burndown Chart:

A burn down chart is a graphical representation of work left to do versus time. It is often used in agile software development methodologies such as Scrum. However, burn down charts can be applied to any project containing measurable progress over time.

For this project the ideal burndown starts at the total committed story points (≈ 41) at the beginning of Sprint-1 and reaches zero at the end of Sprint-3 / hardening sprint. Daily progress is tracked against the average velocity of ~3 points per day.

<https://www.visual-paradigm.com/scrum/scrum-burndown-chart/>

<https://www.atlassian.com/agile/tutorials/burndown-charts>

Reference:

<https://www.atlassian.com/agile/project-management>

<https://www.atlassian.com/agile/tutorials/how-to-do-scrum-with-jira-software>

<https://www.atlassian.com/agile/tutorials/epics>

<https://www.atlassian.com/agile/tutorials/sprints>

<https://www.atlassian.com/agile/project-management/estimation>

<https://www.atlassian.com/agile/tutorials/burndown-charts>

<https://developer.servicenow.com/>